

Rakenduste programmeerimine 1. kodutöö

Martin Sütt

230515IFIB.DT

1. Videote kokkuvõtted

1.1. (What the heck is the event loop anyway? | Philip Roberts | JSConf EU -

<https://www.youtube.com/watch?v=8aGhZQkoFbQ>)

See video rääkis *call stack*'ist, Chrome'i V8 mootorist, sünkroonses vs. asünkroonses koodist, Web API-dest, *task queue*'st ja *event loop*'ist - ning sellest, kuidas kõik need asjad omavahel koos töötavad, et JavaScript (olles ise tegelt *single-threaded* programmeerimiskeel) suudaks hakkama saada ka aeganõudvate asjadega ilma et brauseri leht kinni jookseks.

Sain kinnistust oma teadmiste selle osas, mida *call stack* tähendab - see on nagu virn, kuhu JavaScript paneb asju, mis tuleb ära teha. Iga funktsioonikutsega tuleb midagi selle virna otsa, ja kui jõuame selle funktsiooni *return*-ini, võetakse see jälle virnast maha. Enne ei saa järgmise asja juurde liikuda, kui eelmine on täidetud ja *stack*'ilt maas.

Sain teada, mis on V8 - see on lihtsalt Chrome'i (ja Node'i) sees olev mootor, mis JavaScript'i koodi käivitab. Üllatav oli teada saada, et asjad nagu *setTimeout* või HTTP-päringud ei tulegi JavaScriptist endast, vaid on hoopis brauseri pakutavad Web API-d. Seetõttu saabki JavaScript ise jääda *single-threaded*'iks (ehk teeb ühe asja korraga) ega blokeeru, samal ajal kui brauser tegeleb aeganõudvate asjadega taustal. Õppisin ka, et mis asi on see *event loop* - see on selline lihtne mehhanism, mille ainus ülesanne on vaadata, kas *call stack* on tühi või ei. Kui on tühi, siis võtab ta *task queue*'s ootavatest *callback*'idest esimese ja tõstab selle *stack*'ile, mis käivitab selle täpselt samas järjekorras, milles need *queue*'sse jõudsid. Tänu sellele sain ka aru sellest, miks *setTimeout(0)* töötab nii nagu ta töötab: isegi nullise viivitusega *callback* peab enne käivitumist ikkagi ootama *task queue*'s kuni *stack* saab tühjaks, mistõttu seda kasutataksegi, et panna mingi osa koodist käivituma alles pärast sünkroonset koodi.

1.2. Asynchrony: Under the Hood - Shelley Vohr - JSConf EU -

(<https://www.youtube.com/watch?v=SrNQS8J67zc>)

See video rääkis *call stack*'ist, *task queue*'st, *event loop*'ist, *callback*'idest, *promise*'itest ja *microtask queue*'st - keskendudes rohkem sellele, mis probleemid *callback*'idega kaasnevad ja kuidas *promise*'itega neid lahendada saab.

Sain teada, mis on *callback hell* - kui *callback*'e on palju ja need on üksteise sees, muutub kood raskesti loetavaks, kuna inimene tahaks lugeda ülevalt alla, kuid *callback*'id ei jookse selles järjekorras. Lisaks tekib ilma *promise*'iteta "*inversion of control*" - kui annad oma *callback*'i mõnele teisele funktsioonile, kaotad kontrolli selle üle, millal või kas üldse see *callback* õigesti käivitatakse.

Õppisin, et *promise* on justkui kohahoidja tuleviku väärtuse jaoks - see on objekt, mille asünkroonne funktsioon (nt `getUser(id)`) tagastab kohe *promise*'i, isegi kui tegelik väärtus pole veel teada. `.then()` on koht, kus saab määrata, mida teha siis, kui see väärtus lõpuks valmis saab (nt `getUser(id).then(user => console.log(user))`). *Promise*'eid saab ka omavahel ahelasse panna (`.then()` peale `.then()` jne.), et kujutada mitmest sammust koosnevat asünkroonset tegevust.

Lisaks sain teada, et *promise*'id kasutavad tavalise *task queue* asemel *microtask queue*'d - kui *task queue*'s olev ülesanne on lõpetatud, tehakse kõigepealt kõik ootel olevad *microtask queue*'s olevad ülesanded, ja alles siis läheb *event loop* edasi järgmise *task queue*'s oleva ülesande juurde.

Samuti sain teada *generator* funktsioonide (`function*`, `yield`) kohta - funktsioon, mida saab käivitamise keskel peatada ja hiljem sellega (`.next()` abil) jätkata.

Videos oli juttu ka *async/await*-ist, mis toetub klassikalistele *promise*'itele - *async* funktsioon tagastab alati *promise*'i ja *await* peatab funktsiooni täitmise seni, kuni oodatav *promise* on täide viidud, ilma et see samal ajal kogu programmi blokeeriks (ülejäanud kood jookseb edasi).

1.3. Javascript Promises vs Async Await EXPLAINED (in 5 minutes)

(<https://www.youtube.com/watch?v=li7FzDHYZpc>)

See video näitas *promise*'ite ja *async/await*'i kasutamist läbi konkreetse näite (API-päring *axios* library ja *boredapi.com* abil), mis aitas mul eelmiste videote teooriat praktikas paremini mõista. Video käis üle selle, kuidas programm ei jää

promise'i taha kinni, vaid jookseb edasi ja ainult `.then()` / `.catch()` sisse kirjutatud kood ootab tulemust.

Samuti sain selgemaks, kuidas veakäsitlus kahe süntaksi vahel erineb: `.then()` / `.catch()` puhul püütakse vead kinni `.catch()` meetodiga, aga `async/await`'i puhul pannakse kood lihtsalt tavalisse `try/catch` plokki, samamoodi nagu tavalise sünkroonse koodi puhul.

2. ja 3. ülesanded

<https://github.com/martsutt/rakenduste-programmeerimine-2026>